

Introdução ao Kotlin

Desenvolvimento de Software para Dispositivos Móveis

Prof. Me. Sergio Souza Novak — UniRV

31 de julho de 2026



Sumário

- 1 O que é Kotlin?
- 2 Características da Linguagem
- 3 Execução de Programas
- 4 Primeiro Hello World em Kotlin
 - Criando e Configurando o Projeto
 - Estrutura e Execução
 - Projetos Gradle e Pacotes
- 5 Fundamentos da Linguagem
 - Variáveis: val vs var
 - Tipos de Dados e Null Safety
- 6 Funções em Kotlin
 - Declaração de Funções
 - Expressões Lambda e Sintaxe
- 7 Orientação a Objetos em Kotlin
 - Classes, Objetos e Membros
 - Construtores e Inicialização
 - Encapsulamento e Visibilidade
 - Herança, Polimorfismo e Abstração
 - Membros Estáticos (companion object)
- 8 Entrada de Dados, Vetores, Matrizes e Matemática
 - Leitura de Dados do Terminal (Escaneamento)
 - Vetores (Arrays) e Coleções
 - Matrizes (Arrays Multidimensionais)
 - Funções Matemáticas: Seno, Cosseno e Pacote kotlin.math
 - Coleções em Kotlin (Lists, Sets, Maps)

O que é Kotlin?

- Linguagem de programação moderna, estaticamente tipada e de uso geral.
- Criada pela JetBrains em 2011.
- Tornou-se a linguagem oficial para desenvolvimento Android pelo Google em 2017.
- 100% interoperável com Java.
- Código aberto (Open Source).
- Suporta desenvolvimento Android, Web (JS/Wasm), Desktop e Multiplataforma (KMP).



100% compatível com Java

Kotlin **Java**

- **Interoperável:** usa classes nativas do Java em Kotlin. ↗
- Orientada a objetos e funcional.
- Estaticamente tipada.
- Concisa.
- **Segura:** tratamento de nulos (Null Safety).



Interoperabilidade — ArrayList Java em Kotlin:

```
import java.util.ArrayList

fun main(args: Array<String>) {
    val lista = ArrayList<String>() // lista Java
    lista.add("Sergio")
    lista.add("Leandro")
    for (item in lista) {
        System.out.println(item) // java
        println(item)           // kotlin
    }
}
```

Null Safety — Kotlin vs. Java:

Kotlin

```
var valor: String? = null
println(valor.equals("Sergio"))
// Resultado: false
```

Kotlin lida com String? de forma segura, retornando false

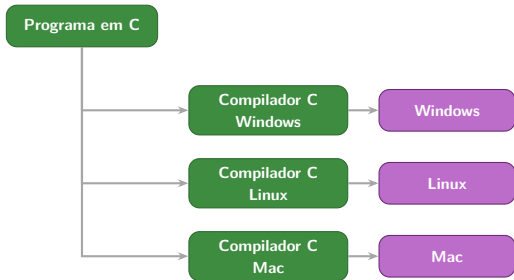
Java

```
String valor = null;
System.out.println(valor.equals("Leandro"))
;
// java.lang.NullPointerException
```

Execução de programas

- Processo para execução de um programa

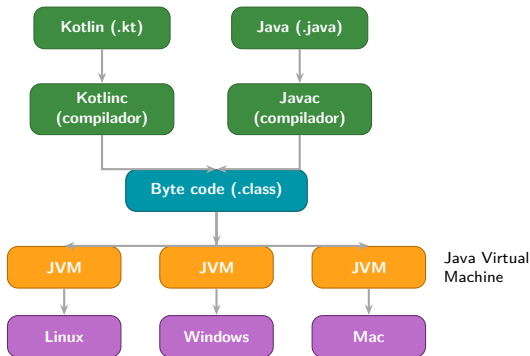
- Escrita
- Compilação
- Execução



- Processo para execução de um programa

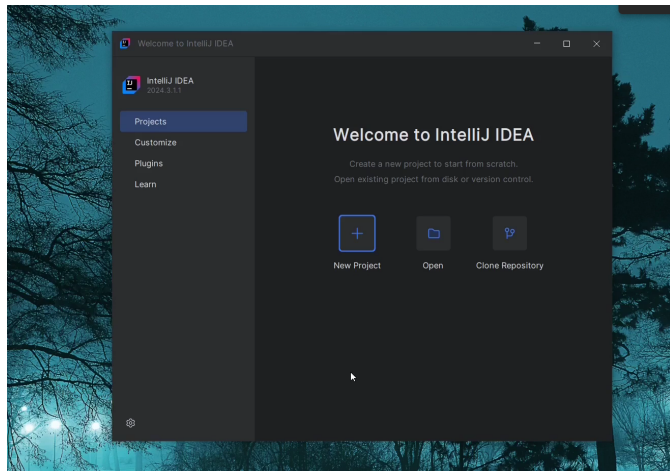
Kotlin/Java

- Escrita
- Compilação
- Execução



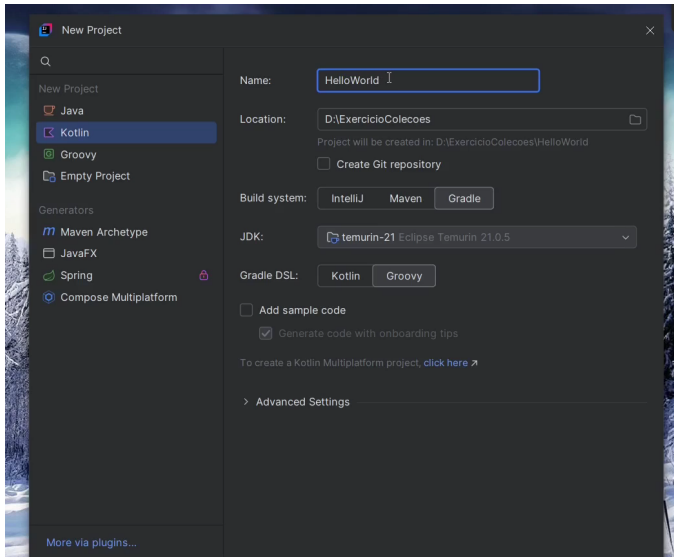
Criando um Novo Projeto no IntelliJ IDEA

- Abra o **IntelliJ IDEA**.
- Na tela inicial de boas-vindas, clique no botão **New Project (+)**.



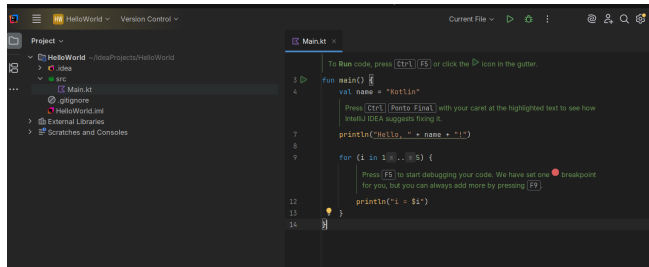
Configurando o Projeto Kotlin

- Selecione **Kotlin** no painel esquerdo.
- Defina o nome do projeto (ex: HelloWorld).
- Escolha o local no disco.
- Selecione o **Build system** (IntelliJ ou Gradle) e a **JDK** desejada.
- Clique em **Create**.



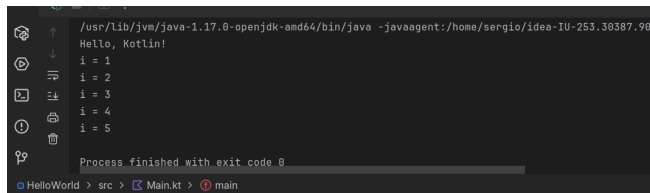
Estrutura do Projeto e Código Inicial

- O arquivo principal é o `Main.kt` dentro da pasta `src`.
- A função principal `fun main()` é o ponto de entrada da aplicação.
- Utilize o ícone de reprodução verde (▶) ao lado da linha para executar o código.



Executando e Analisando o Terminal de Saída

- Ao executar, o painel **Run** é exibido na parte inferior.
- A saída do comando `println()` será impressa no console.
- O código de saída `exit code 0` indica execução com sucesso.

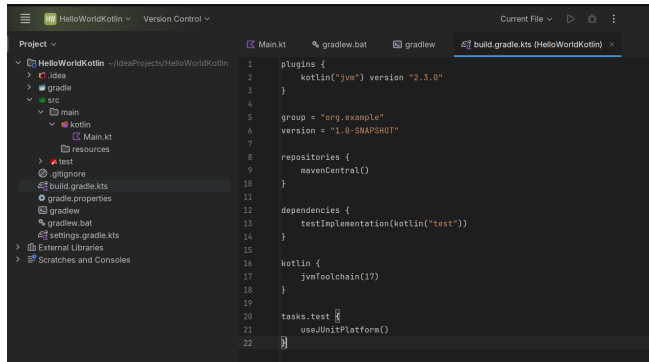


```
/usr/lib/jvm/java-1.17.0-openjdk-amd64/bin/java -javaagent:/home/sergio/idea-IU-253.30387.90
Hello, Kotlin!
i = 1
i = 2
i = 3
i = 4
i = 5
Process finished with exit code 0
```

The screenshot shows the Run console in IntelliJ IDEA. The output of the program is displayed, including the greeting "Hello, Kotlin!" and a loop of numbers from 1 to 5. The console also shows the command used to run the program and the final exit code 0, indicating successful execution. The breadcrumb at the bottom indicates the current file is Main.kt in the main function.

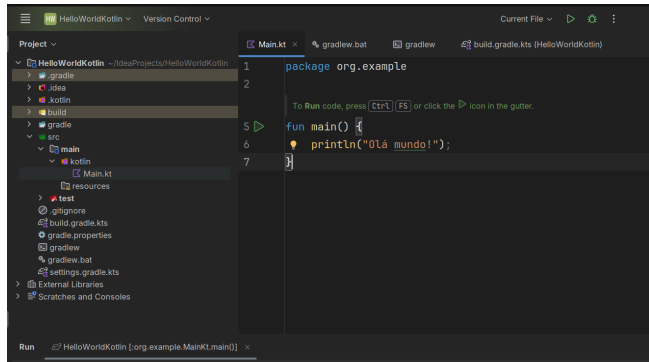
Estrutura com Gradle (build.gradle.kts)

- Em projetos Gradle, o arquivo `build.gradle.kts` define as configurações.
- Declara plugins (como `kotlin("jvm")`), repositórios e dependências.
- O código-fonte fica em `src/main/kotlin/`.



Hello World com Pacotes em Kotlin

- Definição de pacote (package org.example).
- Exemplo simples imprimindo "Olá mundo!".
- Organização padrão para desenvolvimento de projetos profissionais.



Declaração de Variáveis: val vs var

val (Imutável / Somente Leitura)

- **Valor fixo:** Atribuído apenas **uma vez**.
- Semelhante ao final no Java ou const em JS.
- **Recomendado:** Use val por padrão para código mais seguro e previsível.

var (Mutável)

- **Valor reatribuível:** Pode ser alterado ao longo da execução.
- O tipo da variável continua fixo (estaticamente tipado).

Exemplo Prático:

```
fun main() {  
    // Imutavel (val)  
    val cpf = "123.456.789-00"  
    // cpf = "987.654.321-99" // Erro de  
    // Compilacao!  
  
    // Mutavel (var)  
    var contador = 0  
    contador = 1 // Permitido!  
    contador += 5  
  
    println("CPF: $cpf")  
    println("Contador: $contador")  
}
```

Inteiros e Ponto Flutuante

- **Byte** (8 bits), **Short** (16 bits)
- **Int** (32 bits) — Padrão para inteiros
- **Long** (64 bits) — Sufixo L
- **Float** (32 bits) — Sufixo f ou F
- **Double** (64 bits) — Padrão para decimais

```
fun main() {  
    val idade: Int = 25  
    val habitantes: Long = 214000000L  
    val peso: Float = 70.5f  
    val preco: Double = 199.99  
  
    println("Idade: $idade")  
    println("Habitantes: $habitantes")  
    println("Peso: $peso | Preço: $preco")  
}
```

Booleanos e Caracteres

- **Boolean**: Representa valores lógicos (true ou false).
- Suporta operadores lógicos: &&, ||, !.
- **Char**: Representa um único caractere Unicode envolvido por aspas simples ('').

```
fun main() {  
    val ativo: Boolean = true  
    val maiorDeIdade: Boolean = false  
  
    val genero: Char = 'M'  
    val simbolo: Char = 'A'  
  
    println("Ativo: $ativo")  
    println("Pode dirigir? ${ativo &&  
        maiorDeIdade}")  
    println("Genero: $genero")  
}
```

Tipos de Dados: String e Templates

Cadeias de Caracteres

- **String**: Sequência de caracteres entre aspas duplas ().
- **String Templates**: Interpolação de variáveis usando \$nome.
- Expressões complexas podem ser avaliadas usando \$expressao.
- Suporta Strings de múltiplas linhas com três aspas ().

```
fun main() {  
    val nome = "Sergio"  
    val sobrenome = "Novak"  
  
    // String Template  
    val nomeCompleto = "$nome $sobrenome"  
    val mensagem = "Tamanho: ${nomeCompleto.  
        length}"  
  
    println(nomeCompleto)  
    println(mensagem)  
}
```

Tipos Anuláveis (Nullable Types)

Null Safety no Sistema de Tipos

- Por padrão, os tipos em Kotlin **não aceitam nulo**.
- Adiciona-se o operador `?` para permitir valores nulos (`Tipo?`).
- Chamada segura: operador `?.`
- Operador Elvis: `?:` para fornecer um valor padrão.

```
fun main() {  
    var apelido: String? = null  
  
    // Chamada segura (retorna null se for  
        null)  
    println(apelido?.length)  
  
    // Operador Elvis (retorna valor padrao)  
    val exibicao = apelido ?: "Sem apelido"  
    println(exibicao)  
}
```

Formas de Declarar Funções

- **Funções de Expressão Única:**

- Sintaxe concisa usando o operador =.
- Opcional especificar o tipo de retorno quando o compilador infere.

- **Funções com Corpo em Bloco:**

- Usam chaves { } e exigem o comando return (se houver retorno).
- Requerem definição explícita do tipo de retorno.

```
// Expressao de linha unica (Single-expression)
fun helloWorld(nome: String) =
    println("Ola, $nome!")

// Corpo em bloco com retorno explicito
fun media(n1: Int, n2: Int): Int {
    val media = (n1 + n2) / 2
    return media
}

fun main() {
    helloWorld("sergio")
    println(media(10, 8))
}
```

Funções Lambda (Sintaxe com Chaves {})

Expressões Lambda em Kotlin

- Forma comum de escrever *arrow functions*.
- Toda a expressão fica contida dentro de chaves { }.
- Usa a seta simples -> para separar parâmetros do corpo (em vez de =>).

Comparativo: JavaScript vs. Kotlin

```
// JavaScript  
const soma = (a, b) => a + b;
```

```
// Kotlin  
val soma = { a: Int, b: Int -> a + b }  
  
fun main() {  
    println(soma(5, 3)) // Saída: 8  
}
```

O Parâmetro Implícito it

Simplificação para Parâmetro Único

- Se a função lambda receber apenas um parâmetro, você não precisa declarar a seta ->.
- O Kotlin disponibiliza automaticamente o nome `it` para se referir a esse parâmetro.
- Torna o código muito mais legível em operações com coleções (ex: `map`, `filter`).

```
// Em vez de: { x: Int -> x * 2 }  
val numeros = listOf(1, 2, 3)  
  
// Usando o parametro implicito 'it':  
val dobrados = numeros.map { it * 2 }  
  
fun main() {  
    println(dobrados) // [2, 4, 6]  
}
```

Funções de Expressão Única (Single-Expression)

Funções de Linha Única com =

- Declaradas com a palavra-chave `fun`.
- Quando possuem apenas uma linha de retorno, omite-se o bloco `{ }` e a palavra `return`.
- Utiliza-se apenas o operador `=`.
- O tipo de retorno é inferido automaticamente pelo compilador.

```
// Funcao tradicional  
fun helloTradicional(nome: String): Unit {  
    println("Ola, $nome!")  
}  
  
// Estilo de expressao unica  
fun helloWorld(nome: String) =  
    println("Ola, $nome!")  
  
// Exemplo com retorno de valor  
fun dobro(n: Int) = n * 2
```

Resumo das Diferenças de Sintaxe

Conceito	JavaScript	Kotlin
Seta utilizada	=>	->
Escopo da Lambda	(a, b) => a + b	{ a, b -> a + b }
Parâmetro único	x => x * 2	{ it * 2 }
Expressão única em função	<i>Não possui suporte nativo</i>	fun dobro(n) = n * 2

O que é uma Classe?

- Uma classe é um **modelo ou molde** para criação de objetos.
- Ela ainda **não ocupa memória** por si só.
- Apenas define a estrutura e comportamento que os objetos terão.

Sintaxe Básica:

```
class Pessoa {  
    // Corpo da classe (opcional se vazia)  
}
```

Instanciando Objetos

- Objetos são **instâncias reais** de uma classe.
- Alocam espaço na memória ao serem criados.
- Cada objeto possui sua própria identidade e estado.
- Em Kotlin, **não** se usa a palavra-chave `new`.

```
class Pessoa

fun main() {
    // Criando objetos (instancias)
    val p1 = Pessoa()
    val p2 = Pessoa()

    // p1 e p2 sao objetos distintos
}
```

Características dos Objetos

- Atributos (propriedades) armazenam os dados/estado do objeto.
- Declarados com `var` (mutáveis) ou `val` (imutáveis).
- Acessados e alterados via notação de ponto (`objeto.atributo`).

```
class Pessoa {  
    var nome = ""  
    var idade = 0  
}  
  
fun main() {  
    val pessoa = Pessoa()  
  
    pessoa.nome = "Maria"  
    pessoa.idade = 20  
  
    println("${pessoa.nome} tem ${pessoa.  
        idade} anos")  
}
```

Comportamento dos Objetos

- Métodos são funções declaradas dentro de uma classe.
- Representam as ações/comportamentos que os objetos podem realizar.
- Podem acessar diretamente as propriedades da classe.

```
class Pessoa {  
    var nome = ""  
  
    fun apresentar() {  
        println("Ola, meu nome e $nome")  
    }  
}  
  
fun main() {  
    val p = Pessoa()  
    p.nome = "Carlos"  
    p.apresentar()  
}
```

Sintaxe Enxuta em Kotlin

- O construtor primário faz parte do cabeçalho da classe.
- Declara e inicializa propriedades diretamente no parâmetro.
- Reduz significativamente o código boilerplate.

```
// Declaracao concisa com construtor primario  
class Pessoa(  
    var nome: String,  
    var idade: Int  
)  
  
fun main() {  
    // Instanciando e inicializando direto:  
    val p = Pessoa("Ana", 22)  
    println("${p.nome}, ${p.idade} anos")  
}
```

Múltiplas Formas de Construção

- Declarados com a palavra `constructor`.
- Permitem fornecer construtores alternativos (ex: sem parâmetros e com parâmetros).
- Usam a referência `this` para atribuir aos atributos.

```
class Pessoa {  
    var nome = ""  
    var idade = 0  
  
    constructor()  
  
    constructor(nome: String, idade: Int) {  
        this.nome = nome  
        this.idade = idade  
    }  
}
```

Execução Automática

- O bloco `init` é executado automaticamente assim que o objeto é instanciado.
- Útil para validações, logs ou inicializações complexas no construtor primário.

```
class Pessoa(  
    var nome: String,  
    var idade: Int  
) {  
    init {  
        println("Objeto $nome criado com  
                sucesso!")  
    }  
}  
  
fun main() {  
    val p = Pessoa("Lucas", 30)  
}
```

Modificadores de Visibilidade: `public`

Modificador Padrão

- É o modificador de acesso padrão em Kotlin.
- Mesmo que você não escreva a palavra `public`, ela é assumida pelo compilador.
- Elementos públicos podem ser acessados de qualquer lugar.

Exemplos Equivalentes:

```
class Pessoa {  
    public var nome = "Maria"  
  
    fun apresentar() {  
        println(nome)  
    }  
}  
  
// Equivalente a:  
class Pessoa {  
    var nome = "Maria"  
  
    fun apresentar() {  
        println(nome)  
    }  
}
```

Restrição ao Escopo Local

- Restringe o acesso estritamente ao próprio arquivo ou classe onde foi declarado.
- Impede que arquivos ou classes externas modifiquem ou leiam o atributo diretamente.
- Tentativas de acesso externo geram erro de compilação.

```
class ContaBancaria {  
    private var saldo = 1000.0  
  
    fun consultarSaldo() {  
        println(saldo)  
    }  
}  
  
fun main() {  
    val conta = ContaBancaria()  
    // Erro de Compilacao!  
    // println(conta.saldo)  
}
```

Modificadores de Visibilidade: protected

Acesso por Subclasses

- Só pode ser acessado pela própria classe e por suas subclasses (classes que a herdam).
- **Não é acessível** de forma externa através da instância da subclasse.

```
open class Animal {  
    protected var energia = 100  
}  
  
class Cachorro : Animal() {  
    fun brincar() {  
        energia -= 10 // Permitido!  
    }  
}  
  
fun main() {  
    val c = Cachorro()  
    // c.energia // Erro de Compilacao!  
}
```

Visibilidade no Mesmo Módulo

- Modificador exclusivo da linguagem Kotlin.
- Permite acesso apenas dentro do **mesmo módulo** de compilação (ex: mesmo projeto Gradle ou módulo IntelliJ).
- Se a classe estiver em uma biblioteca, outro projeto que usar essa biblioteca não poderá acessá-la.

```
internal class BancoDados {  
    fun conectar() {  
        println("Conectado")  
    }  
}  
  
// Visível em todo o módulo atual,  
// porém oculto para outros módulos/  
projetos.
```

Comparativo de Visibilidade

```
open class Pessoa {  
    public var nome = "  
        Ana"  
    private var cpf = "  
        123456"  
    protected var idade =  
        20  
    internal var univ = "  
        UniRV"  
}
```

Membro	Mesma classe	Subclasse	Mesmo módulo	Outro módulo
public	Sim	Sim	Sim	Sim
private	Sim	Não	Não	Não
protected	Sim	Sim	Não	Não
internal	Sim	Sim	Sim	Não

Diferença Importante em Relação ao Java

Atenção: Ausência de `package-private`

Diferentemente de linguagens como Java, o Kotlin **não possui** o modificador de acesso *package-private* (visibilidade restrita ao mesmo pacote).

Em Kotlin, o equivalente mais próximo para compartilhar código dentro de um projeto é o modificador **`internal`**, cuja visibilidade é baseada no **módulo** (compilação), e não no pacote.

Acesso e Validação de Propriedades

- Kotlin gera automaticamente getters e setters padrão.
- É possível personalizá-los diretamente abaixo da propriedade.
- Usa-se `field` (field de apoio) dentro do getter/setter para evitar recursão infinita.

```
class Pessoa {  
    var idade = 0  
        set(value) {  
            if (value >= 0) {  
                field = value  
            }  
        }  
}  
  
fun main() {  
    val p = Pessoa()  
    p.idade = -5 // Não altera  
    println(p.idade) // Saída: 0  
}
```

Reaproveitamento de Código

- Em Kotlin, todas as classes são `final` por padrão (não podem ser herdadas).
- Deve-se usar a palavra-chave `open` para permitir herança.
- A sintaxe de herança usa dois pontos (`:`) `SuperClasse()`.

```
// Classe base aberta para heranca  
open class Animal {  
    fun dormir() {  
        println("Dormindo...")  
    }  
}  
  
// Classe filha herdando de Animal  
class Cachorro : Animal()
```

Sobrescrita de Métodos (override)

Redefinindo Comportamentos

- O método da classe pai também precisa ser marcado com open.
- A classe filha usa a palavra-chave override para redefinir a implementação.

```
open class Animal {  
    open fun emitirSom() {  
        println("Som generico")  
    }  
}  
  
class Cachorro : Animal() {  
    override fun emitirSom() {  
        println("Au Au")  
    }  
}
```

Múltiplas Formas

- Permite tratar objetos de classes filhas como se fossem do tipo da classe pai.
- O método executado é o da classe instanciada em tempo de execução.
- *Mesmo tipo genérico produzindo comportamentos diferentes.*

```
fun main() {  
    // Tipo da superclasse, instancia da  
    // filha  
    val animal: Animal = Cachorro()  
  
    animal.emitirSom()  
    // Resultado: "Au Au"  
}
```

Modelos Incompletos

- Declaradas com `abstract`.
- Não podem ser instanciadas diretamente.
- Podem ter métodos abstratos (sem corpo) que obrigam a classe filha a implementar.

```
abstract class Forma {  
    abstract fun desenhar()  
}  
  
class Circulo : Forma() {  
    override fun desenhar() {  
        println("Desenhando um circulo")  
    }  
}
```

Contratos de Comportamento

- Definem contratos de ações que uma classe deve cumprir.
- Uma classe pode implementar múltiplas interfaces.
- Métodos em interfaces são abstratos por padrão.

```
interface Voador {  
    fun voar()  
}  
  
class Passaro : Voador {  
    override fun voar() {  
        println("Voando...")  
    }  
}
```

Comparativo: Herança × Interface

Característica	Herança	Interface
Relacionamento	"É um"(ex: Cachorro <i>é um</i> Animal)	"Pode fazer"(ex: Passaro <i>pode</i> Voar)
Quantidade	Apenas uma superclasse	Múltiplas interfaces
Propósito	Compartilha código/implementação	Compartilha contrato de comportamento

Exemplo Completo de Orientação a Objetos

Hierarquia de Funcionários

- Superclasse Funcionario aberta.
- Subclasse Professor sobrescrevendo a lógica de salário baseada nas horas.
- Integração de Construtores, Herança e Sobrescrita.

```
open class Funcionario(var nome: String) {  
    open fun salario(): Double = 0.0  
}  
  
class Professor(  
    nome: String,  
    var horas: Int  
) : Funcionario(nome) {  
    override fun salario(): Double {  
        return horas * 80.0  
    }  
}  
  
fun main() {  
    val prof = Professor("Joao", 40)  
    println(prof.salario()) // 3200.0  
}
```

Membros Estáticos (companion object)

Membros de Classe em Kotlin

- Kotlin não possui a palavra-chave `static`.
- Em seu lugar, utiliza-se o bloco `companion object` dentro da classe.
- Permite acessar atributos e métodos diretamente pela classe, sem precisar instanciá-la.
- Útil para constantes globais, métodos utilitários e padrão *Factory*.

```
class Matematica {  
    companion object {  
        // Atributo "estatico"  
        val PI = 3.14159  
  
        // Metodo "estatico"  
        fun dobro(n: Int): Int {  
            return n * 2  
        }  
    }  
}  
  
fun main() {  
    // Acesso direto pela classe:  
    println(Matematica.PI)  
    println(Matematica.dobro(5)) // 10  
}
```

Lendo Dados do Usuário

- `readln()`: Função nativa e moderna do Kotlin (substituiu o antigo `readLine()`!!).
- Retorna sempre uma `String`.
- É necessário converter o tipo retornado usando métodos como `.toInt()`, `.toDouble()`, etc.
- Também é possível usar a classe `Scanner` da biblioteca Java (`java.util.Scanner`).

```
import java.util.Scanner

fun main() {
    // 1. Usando a funcao nativa readln()
    print("Digite seu nome: ")
    val nome = readln()

    print("Digite sua idade: ")
    val idade = readln().toInt()

    print("Digite sua nota: ")
    val nota = readln().toDouble()

    // 2. Usando a classe Scanner do Java
    val scanner = Scanner(System.`in`)
    print("Digite o peso: ")
    val peso = scanner.nextDouble()

    println("Nome: $nome | Idade: $idade")
}
```

Manipulação de Vetores

- Criados com `arrayOf(...)` ou tipos primitivos dedicados como `IntArray`, `DoubleArray`.
- Acesso por índice via colchetes: `vetor[0]`.
- Propriedades e funções úteis: `.size`, `.sort()`, `.forEach { }`.

```
fun main() {  
    // Vetor de inteiros  
    val numeros = intArrayOf(50, 10, 30,  
        20, 40)  
  
    // Acessando e alterando valores  
    println("Primeiro: ${numeros[0]}")  
    numeros[0] = 99  
  
    // Ordenacao de vetor  
    numeros.sort()  
  
    // Percorrendo o vetor  
    for (n in numeros) {  
        print("$n ")  
    }  
}
```

Matrizes (Arrays 2D)

- Uma matriz é um vetor de vetores: `Array<Array<Tipo>>`.
- Acesso por dois índices: `matriz[linha][coluna]`.
- Inicializadas com `Array(linhas) { Array(colunas) { valor } }` ou listas de vetores.

```
fun main() {  
    // Criando uma matriz 2x3 de Ints  
    val matriz = arrayOf(  
        intArrayOf(1, 2, 3),  
        intArrayOf(4, 5, 6)  
    )  
  
    // Acessando elemento (linha 1, coluna 2)  
    println("Valor [1][2]: ${matriz[1][2]}")  
    // 6  
  
    // Percorrendo a matriz com for aninhado  
    for (i in matriz.indices) {  
        for (j in matriz[i].indices) {  
            print("${matriz[i][j]} ")  
        }  
        println()  
    }  
}
```

Funções Matemáticas: Seno e Cosseno

Trigonometria com kotlin.math

- Importe as funções do pacote `kotlin.math.*`.
- `sin(x)` e `cos(x)` recebem o ângulo obrigatoriamente em **radianos**.
- Use `Math.toRadians(graus)` para converter graus para radianos.
- Outras constantes úteis: `PI`, `sqrt()`, `pow()`, `abs()`.

```
import kotlin.math.sin
import kotlin.math.cos
import kotlin.math.PI

fun main() {
    val anguloGraus = 90.0
    // Converter graus para radianos
    val anguloRadianos = Math.toRadians(
        anguloGraus)

    val seno = sin(anguloRadianos)
    val cosseno = cos(anguloRadianos)

    println("Seno de 90 deg: $seno")
        // 1.0
    println("Cosseno de 90 deg: $cosseno")
        // ~0.0
    println("Valor de PI: $PI")
}
```

Coleções em Kotlin: Imutáveis vs Mutáveis

Visão Geral de Coleções

- Kotlin diferencia estritamente coleções **imutáveis** (somente leitura) de **mutáveis** (alteráveis).
- **Imutáveis (Padrão)**: `listOf()`, `setOf()`, `mapOf()`.
- **Mutáveis**: `mutableListOf()`, `mutableSetOf()`, `mutableMapOf()`.
- Métodos para alteração: `.add()`, `.remove()`, `.clear()`.

```
fun main() {  
    // Lista Imutavel (read-only)  
    val frutas = listOf("Maca", "Banana", "Uva")  
    // frutas.add("Laranja") // Erro de Compilacao!  
  
    // Lista Mutavel  
    val nomes = mutableListOf("Sergio", "Ana")  
    nomes.add("Carlos") // Permitido!  
    nomes.remove("Ana")  
  
    println("Frutas: $frutas")  
    println("Nomes: $nomes")  
}
```

Tipos de Coleções: List, Set e Map

List, Set e Map

- **List**: Coleção ordenada com acesso por índice. Aceita elementos duplicados.
- **Set**: Coleção de elementos **únicos** (sem duplicatas). Ordem não garantida.
- **Map**: Par Chave-Valor (chave to valor). Chaves devem ser únicas.

```
fun main() {  
    // Set (nao aceita duplicatas)  
    val codigos = setOf(101, 102, 101, 103)  
    println(codigos) // [101, 102, 103]  
  
    // Map (par Chave -> Valor)  
    val capitais = mapOf(  
        "GO" to "Goiania",  
        "SP" to "Sao Paulo",  
        "DF" to "Brasilia"  
    )  
    println("Capital de GO: ${capitais["GO"]}")  
}
```

Equivalência de Coleções: Java × Kotlin

Conceito / Tipo	Java (Orientado a Objetos)	Kotlin (Imutável / Mutável)
Lista de Leitura	<code>Collections.unmodifiableList(...)</code>	<code>listOf("A", "B")</code> (Leitura)
Lista Mutável (ArrayList)	<code>new ArrayList<String>()</code>	<code>mutableListOf()</code> ou <code>arrayListOf()</code>
Conjunto (Set)	<code>new HashSet<Integer>()</code>	<code>setOf()</code> ou <code>mutableSetOf()</code>
Mapa (Map / HashMap)	<code>new HashMap<String, String>()</code>	<code>mapOf("K"to "V")</code> ou <code>mutableMapOf()</code>

Nota: Por baixo dos panos (na JVM), o Kotlin reutiliza as próprias classes nativas do Java (`java.util.ArrayList`, `HashSet`, `HashMap`), porém aplicando a distinção entre interfaces mutáveis e imutáveis em tempo de compilação.

Sintaxe Lado a Lado: Java × Kotlin

Java (Verboso e Mutável)

```
import java.util.*;

// List / ArrayList
List<String> lista = new ArrayList<>();
lista.add("Item 1");

// Set / HashSet
Set<Integer> conjunto = new HashSet<>();
conjunto.add(10);

// Map / HashMap
Map<String, Integer> mapa = new HashMap<>();
mapa.put("Chave", 100);
```

Kotlin (Conciso e Seguro)

```
// ArrayList Mutavel
val lista = mutableListOf("Item 1")
val lista2 = arrayListOf("Item 1") //
    nativo

// Set (HashSet)
val conjunto = mutableSetOf(10)
val conjunto2 = hashSetOf(10)

// Map (HashMap)
val mapa = mutableMapOf("Chave" to 100)
val mapa2 = hashMapOf("Chave" to 100)
```

Processamento Funcional

- `filter { }`: Filtra elementos por uma condição.
- `map { }`: Transforma elementos de uma coleção.
- `find { }, first(), count(), sorted()`.
- Usam lambdas e o parâmetro implícito `it`.

```
fun main() {  
    val numeros = listOf(1, 2, 3, 4, 5, 6,  
        7, 8)  
  
    // Filtrar apenas pares  
    val pares = numeros.filter { it % 2 ==  
        0 }  
  
    // Mapear (dobrar valores)  
    val dobrados = numeros.map { it * 2 }  
  
    println("Pares: $pares")           // [2,  
        4, 6, 8]  
    println("Dobrados: $dobrados")    // [2,  
        4, 6, ..., 16]  
}
```